

18341 P5 - USB Report

Our final system has layers of communication interface between the host and device which represents the different stages of the USB protocol. The module **USBHost*** in our code is the top level module that dictates the communication flow between host and device. This is done by instantiating the **readWriteFSM**. The **readWriteFSM** starts either a read or write by triggering the signal **fsmStart** and **re**. This is called in the **readData** and **writeData** task within the **USBHost** module.

In addition to the **readWriteFSM**, we also instantiate the module called **usbController** which handles the OUT and IN transactions necessary for a read or write operation to occur. This is done by the output of the **readWriteFSM** using **startIn_n** and **startOut_n**. Thus, the **readWriteFSM** uses these signals to trigger an (OUT & IN) transaction for a read and an (OUT & OUT) transaction for a write. Then, the **usbController** outputs the success of each transaction to the **readWriteFSM** {**successIN**, **successOut**, **doneIn**, **doneOut**} in order to determine whether the READ/WRITE operation was successful or aborted due to timeouts or corrupted data on the transactional level.

Diving into a lower level, the **usbController** then instantiates the following modules that triggers the hardware needed for the OUT/IN transactions to occur - {**inTxnFSM**, **outTxnFSM**, **controlledSender**, **controlledReceiver**}.

The **inTxnFsm** and **outTxnFSM** here control the sender and receiver to send the address and data needed for the READ/WRITE operation. The FSM uses the signals **startSender** and **reset_nIn/reset_nOut** to trigger the sender/receiver modules and synchronize the communication flow between host and device. In addition, the **inTxnFSM** and **outTxnFSMs** here handle the error scenarios such as time out and data corrupted by using the **controlledSender** and **controlledReceiver** to resend DATA0/NAK to the device as necessary.

Attached below are our FSMs and datapath from the highest level to the sender/receiver level.

As with collaboration, we initially worked on specific modules independently such that Arnav worked on CRCs, packetSender, and bus assignments and William worked on NRZI and bit stuffing. Then, we worked together on the handshaking between the modules. However, as the project progressed to the final portion, we worked on implementing & debugging our design together on campus for the upstream FSMs & receiver implementation from host to device.

***bolded** words are signal/module names

FSM HANDSHAKING AND TRANSFERRED VALUES

Read Write FSM

Outputs

- startOut_n, startIn_n : starts out and in transaction FSMs
- isAddr : 1 if address is being sent, seen by packet receiver
- senderData : data to be sent by packet sender
- Success, done : asserted to indicate when a write/read operation is done
- txnType : transaction type for the in/out transaction FSMs

Inputs

- Success, done out, in : success and done inputs from the in and out transaction FSMs
- Re: read enable
- pktReceived : the packet received from the IN transaction

Out Transaction FSM

Inputs

- startOut_n: starts OUT transaction
- doneSender, finishedReading: taking in the status of the sender and receiver
- payloadReceived: payload received from device

Outputs

- SuccessOut, doneOut: tells ReadWrite FSM the status of the transaction
- Start_sender, resetReceiver_n: start sender thread and receiver thread

In Transaction FSM

Inputs

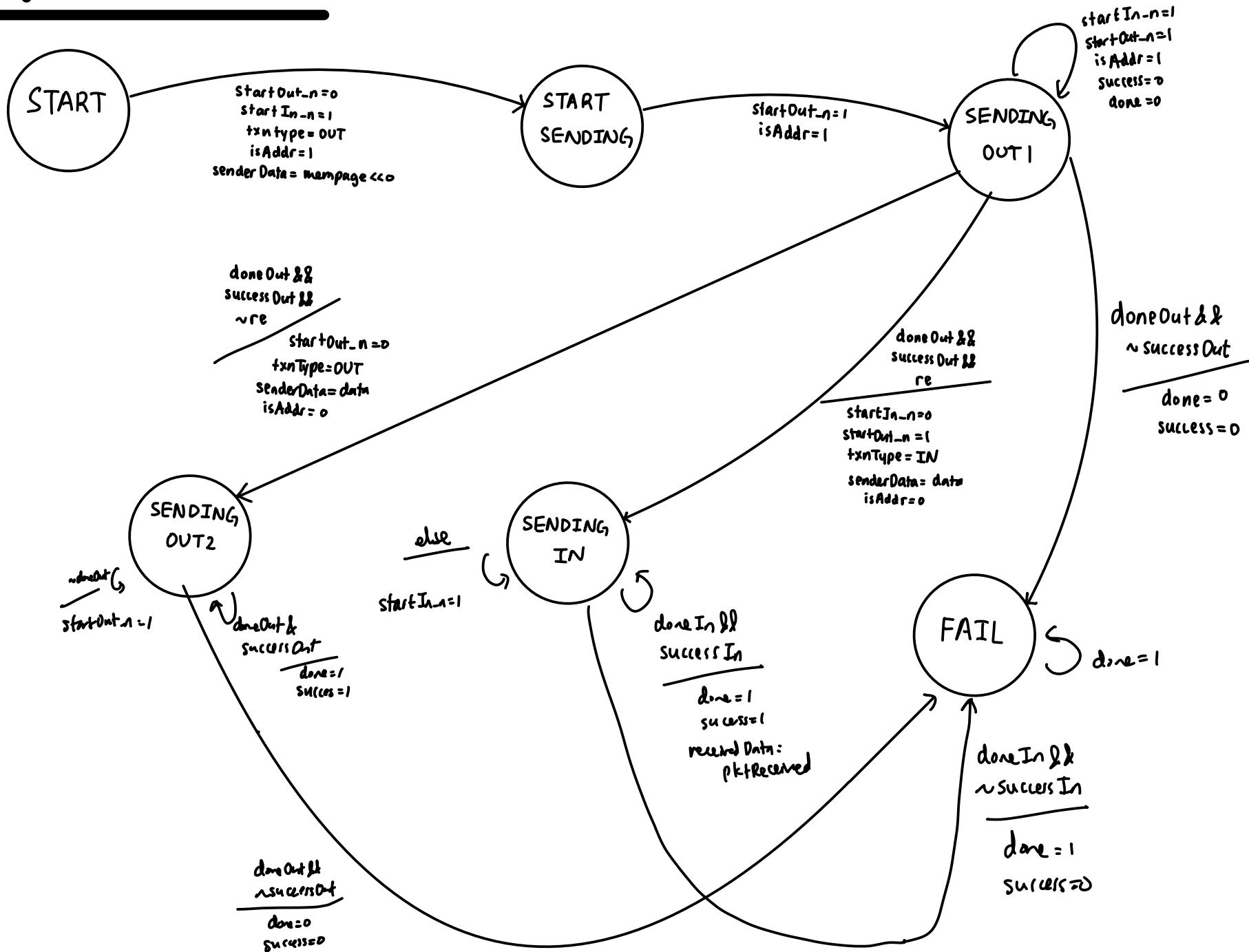
- startIn_n: starts IN transaction
- doneSender : done sending from the packet sender
- finishedReading : when the packet sender is finished sending

Outputs

- SuccessIn, doneIn: tells ReadWrite FSM the status of the transaction
- Start : to start sending a packet
- resetReceiver_n : reset receiver to begin receiving a packet
- Payload : payload seen by packet sender

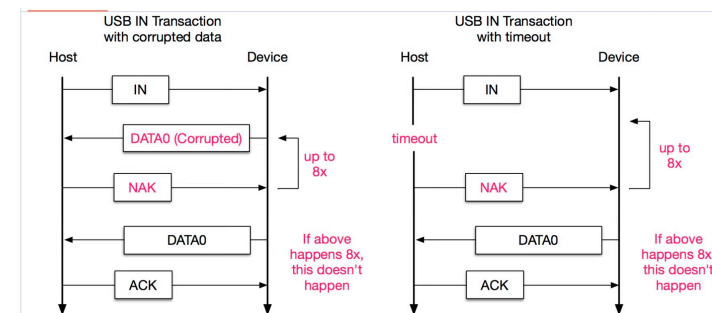
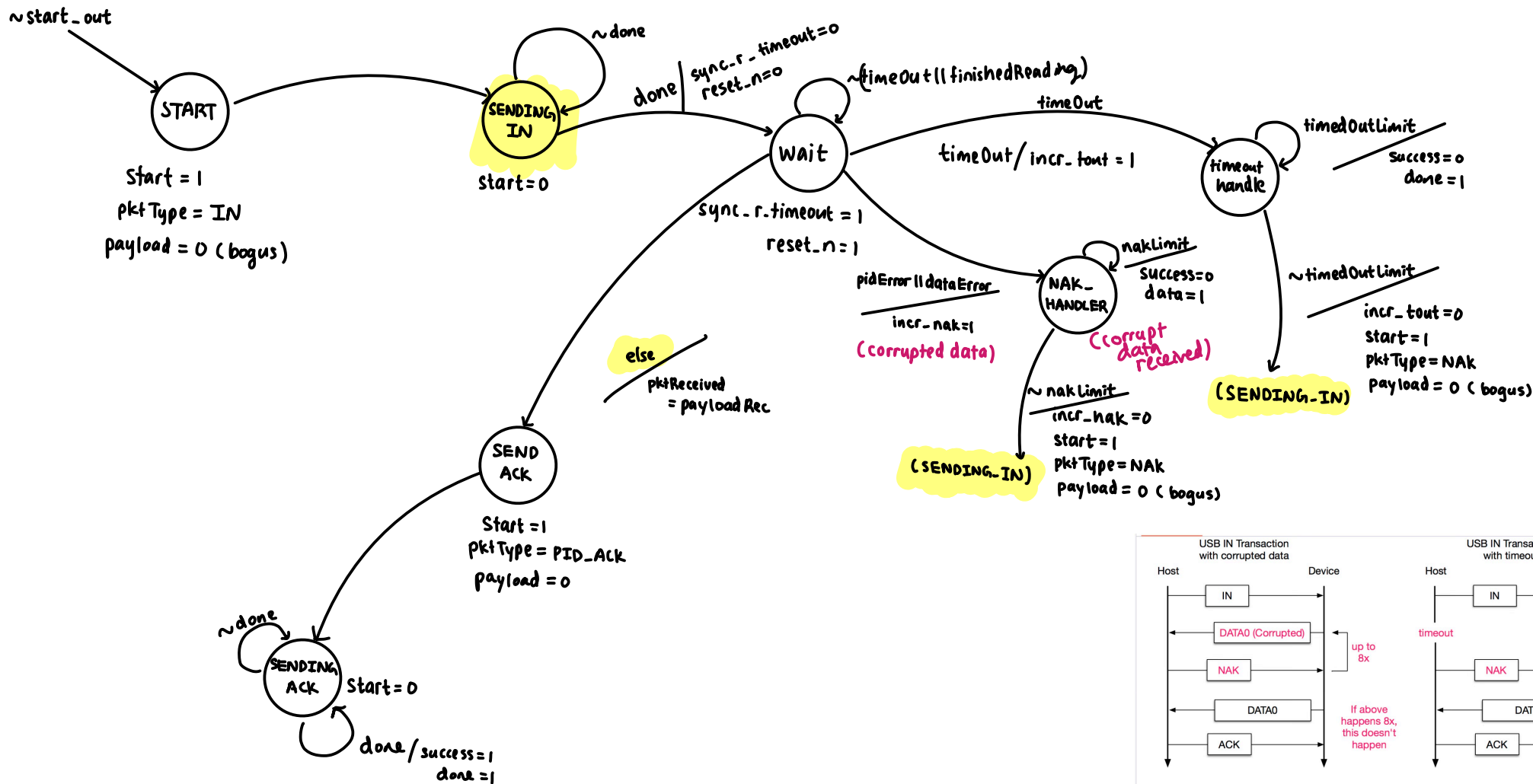
- payloadReceived comes from the receiver
- pktType is sent to the packet sender
- pktTypeReceived seen from the receiver

Read/Write FSM

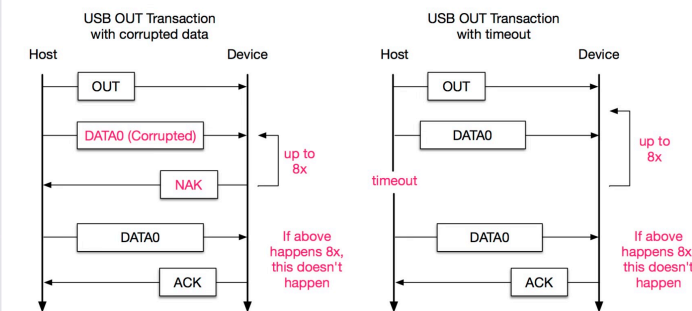


IN TRANSACTION FSM

payload = 64 bits

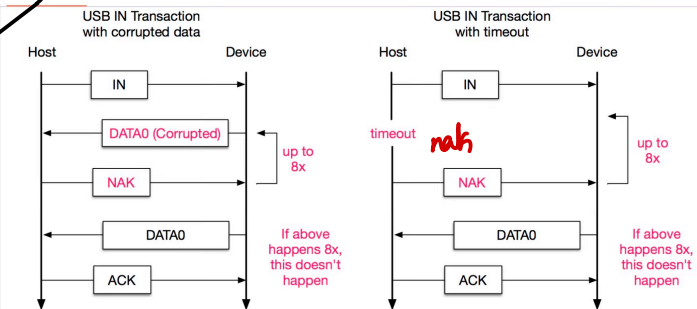
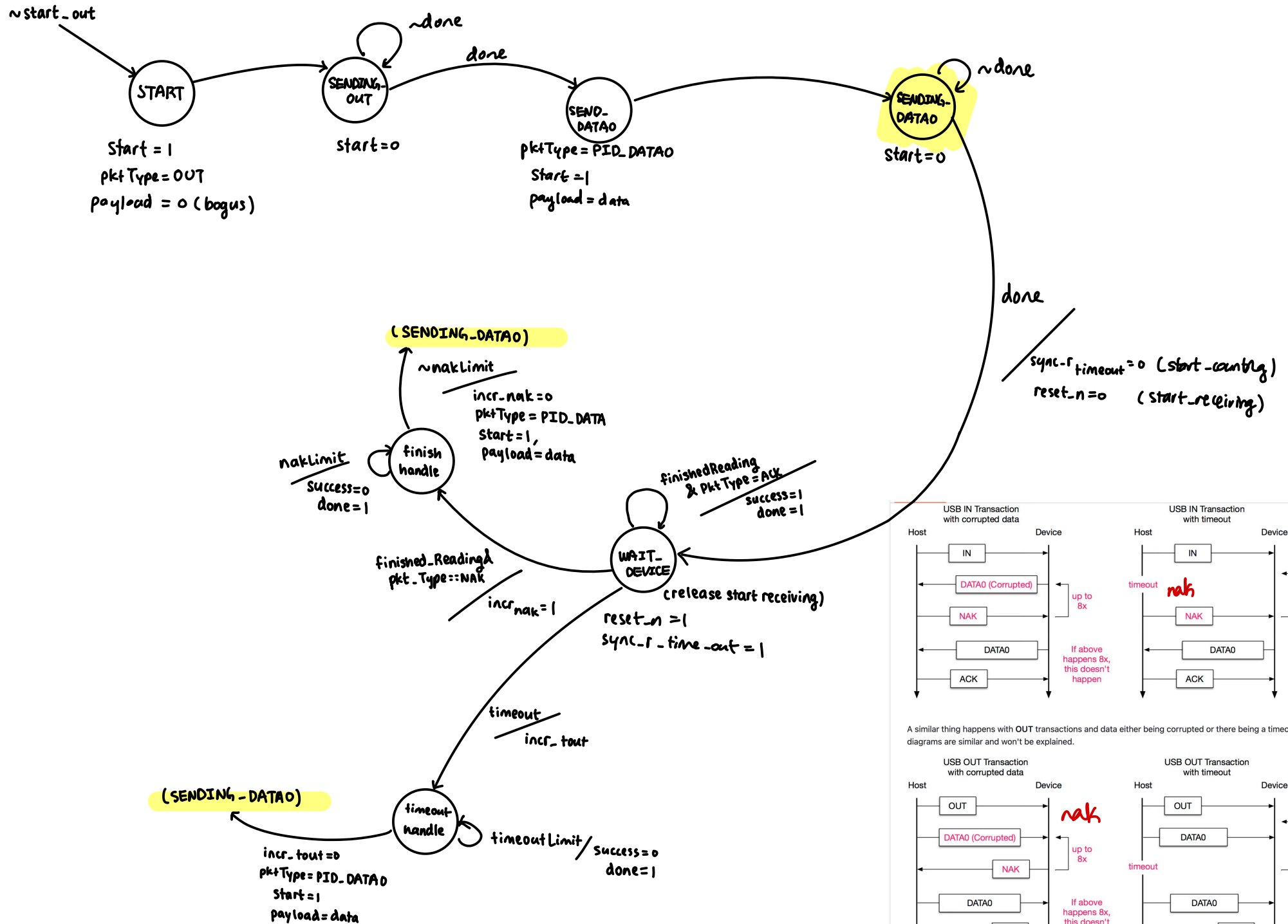


A similar thing happens with OUT transactions and data either being corrupted or there being a timeout. The diagrams are similar and won't be explained.

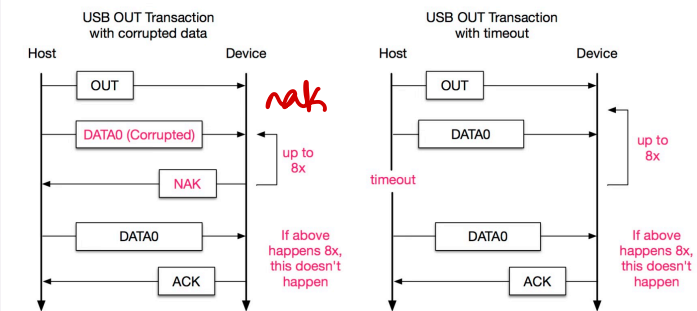


→ handler_mempage
 read = OUT, IN
 write = OUT, OUT
 → write_data
 → handler_mempage

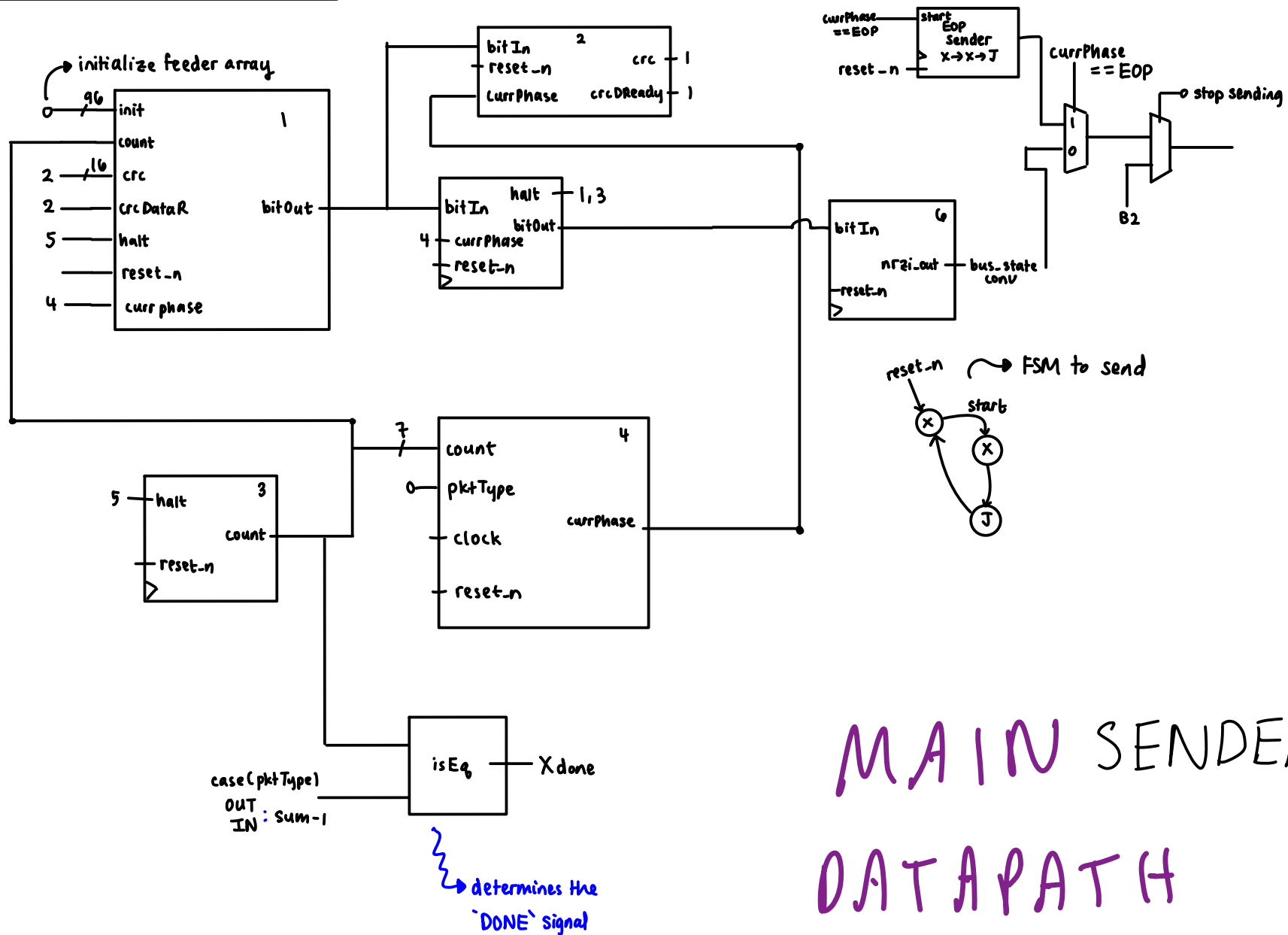
OUT TRANSACTION FSM



A similar thing happens with OUT transactions and data either being corrupted or there being a timeout. The diagrams are similar and won't be explained.

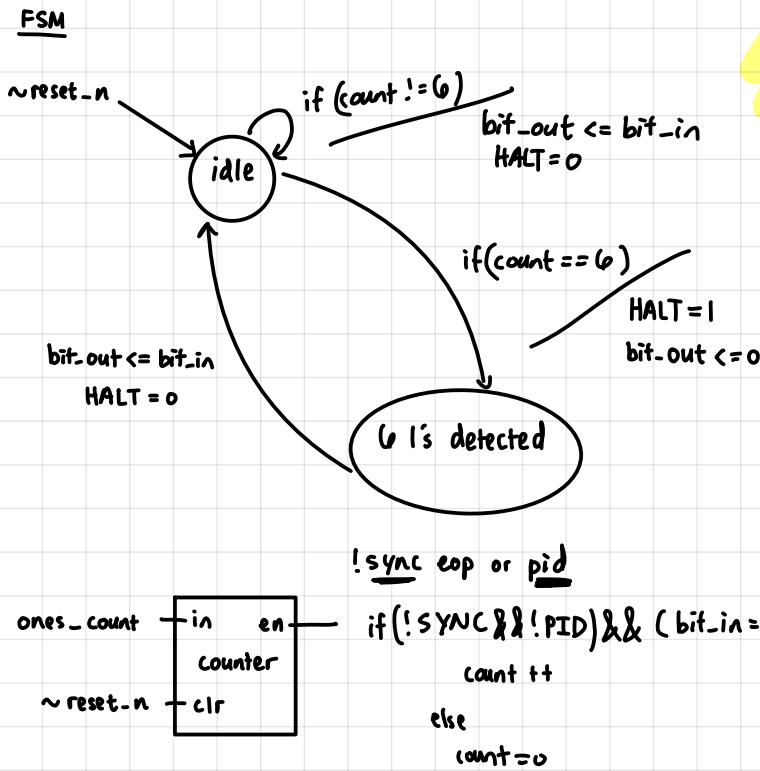


SENDER



MAIN SENDER
DATAPATH

Bit Stuffing



Bit Stuffing

- ↳ 0 inserted after
- ↳ consecutive 1's

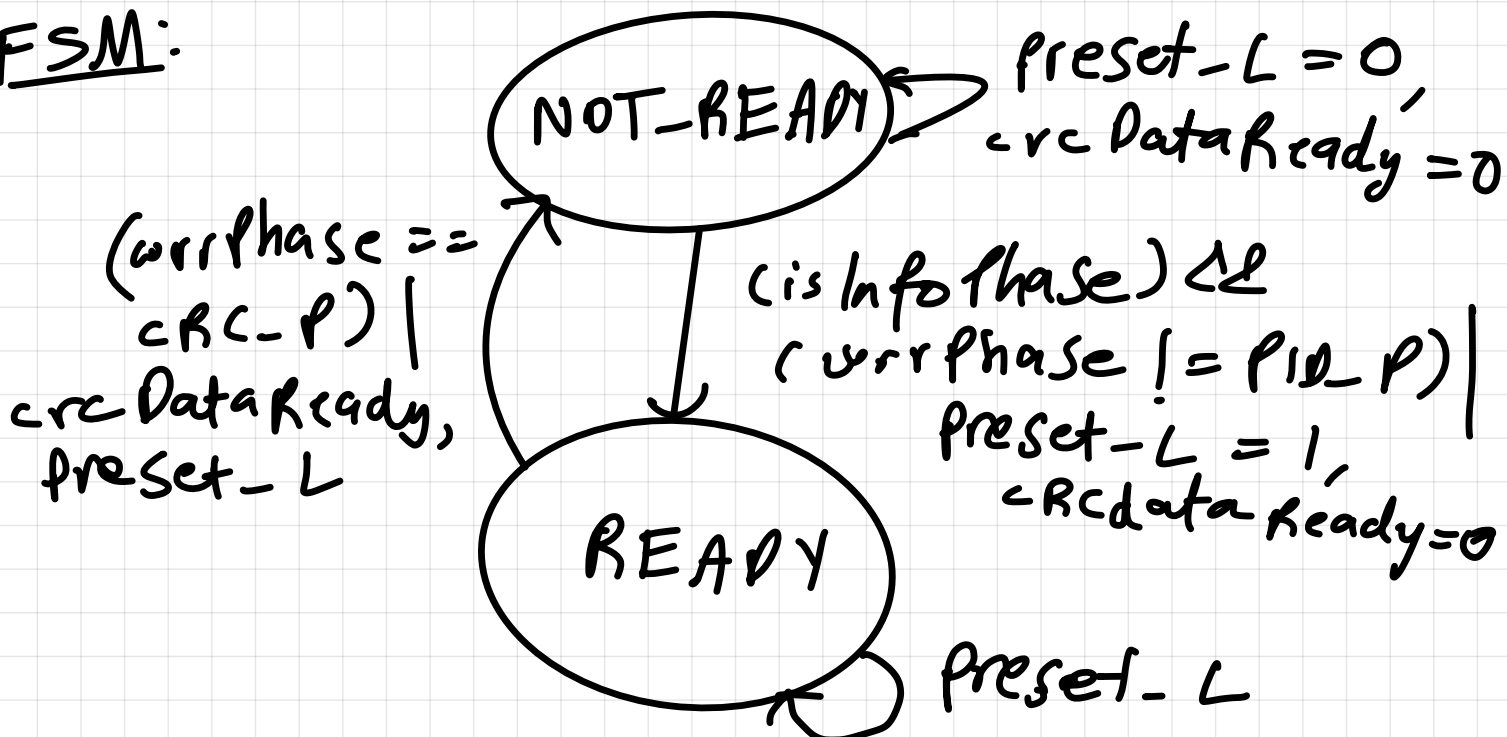
phase determined by enum statements

* SYNC, PID, ADDR, ENDP, DATA, CRC*

CRC Generation.

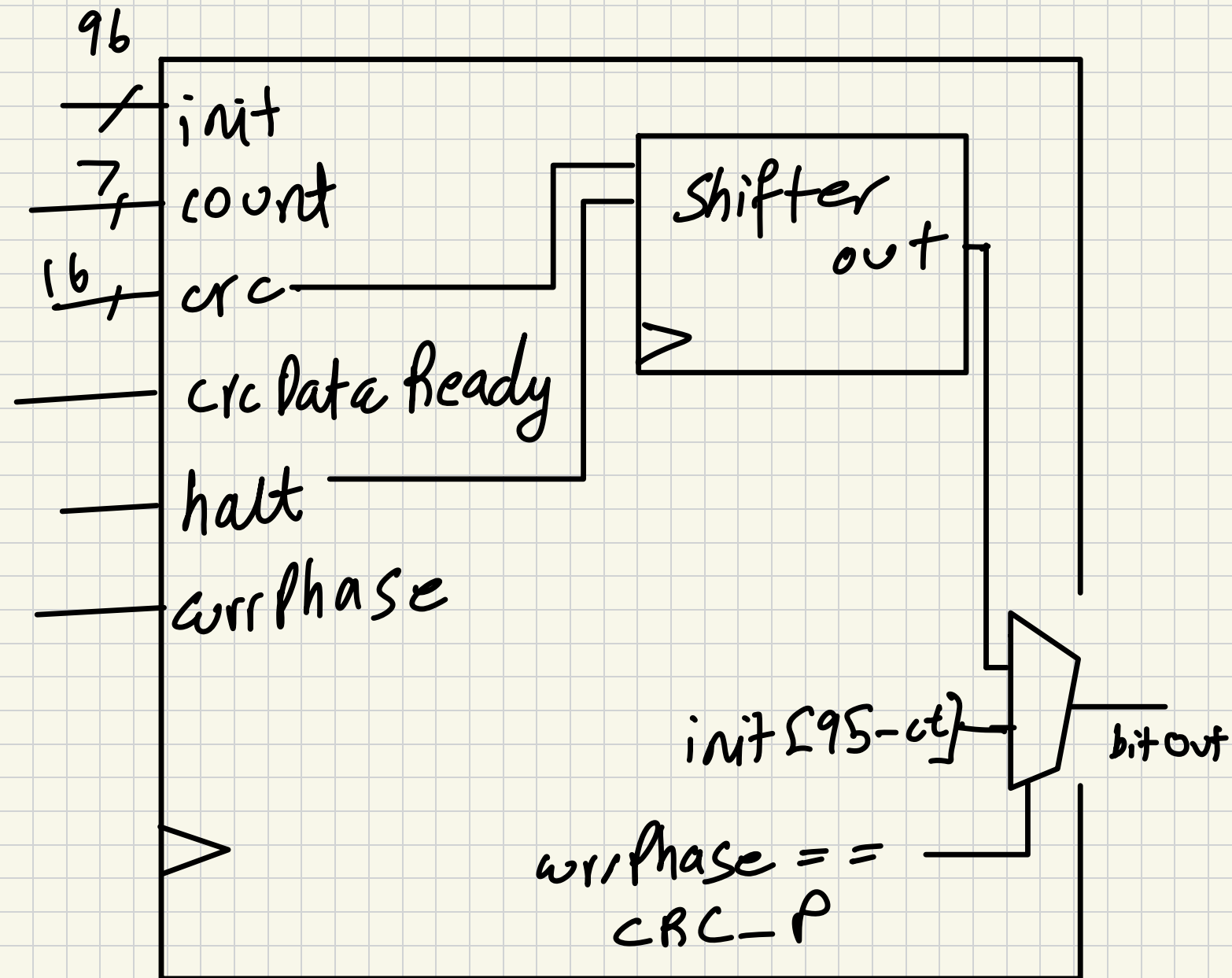
Uses CRC5 Datapath from L15.

FSM:



where
isInfoPhase = 1 when current phase
is PAYLOAD-P, ADDR-P, PID-P,
END-P.

Feeder Array (NO FSM)



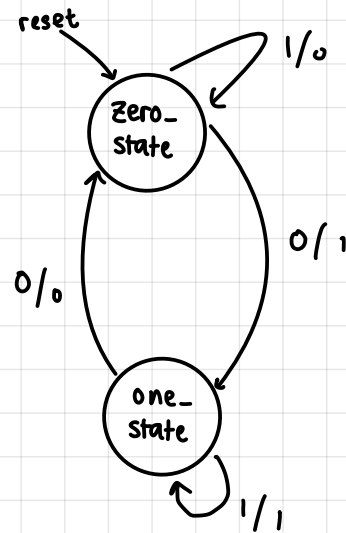
NRZI Encoding

USB NRZI Encoding

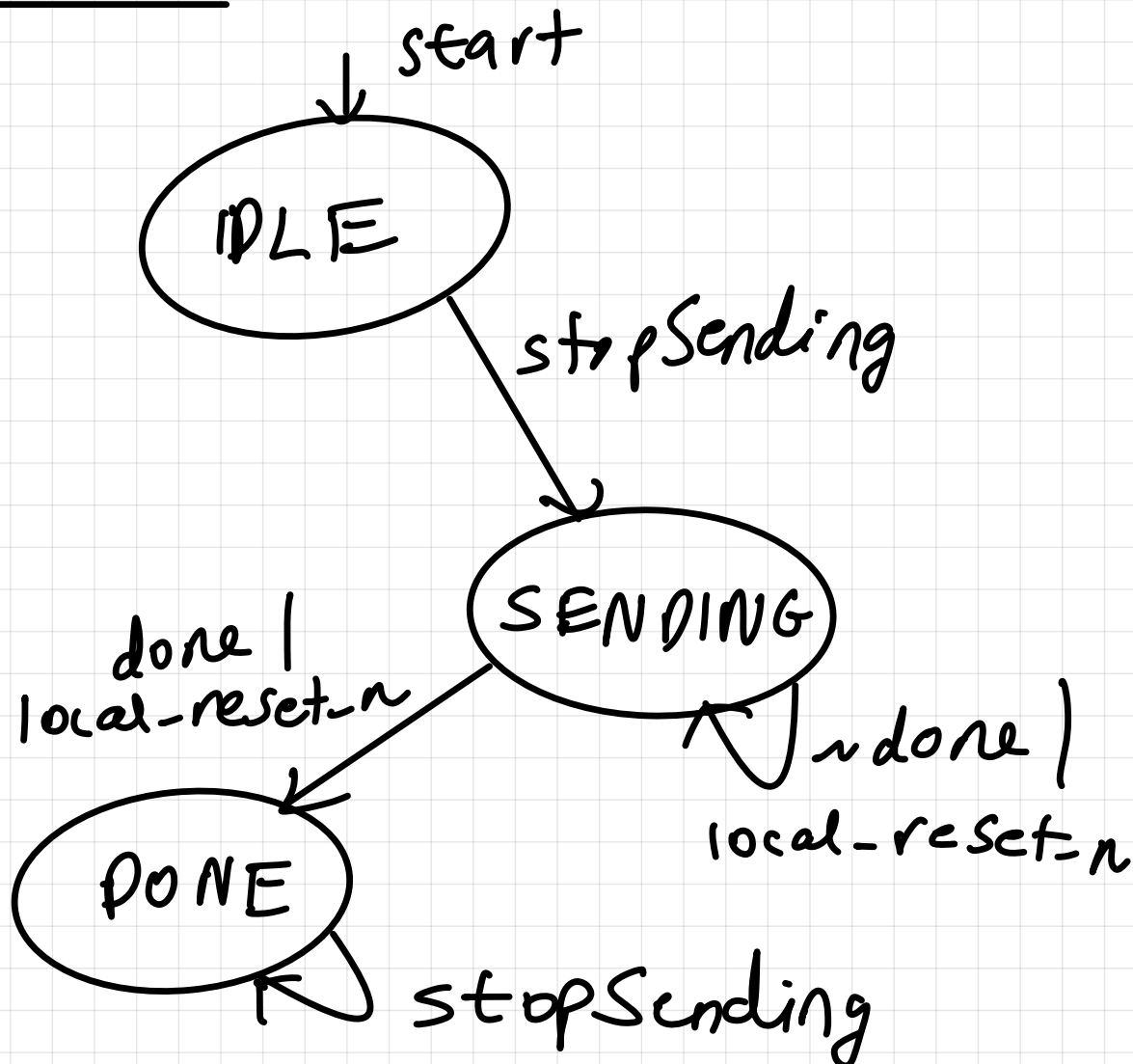
- all field types are sent through NRZI except for EOP

Assume $\text{prev_val} \leq 1$

- 0 $\rightarrow$ toggling output
- 1 $\rightarrow$ leaving output as is

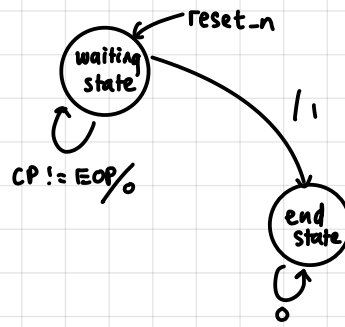


System FSM [send packet Helper]




```

graph TD
    0((0)) -- "reset - n" --> 0
    0 -- "~pid-check" --> 0
    0 -- "pid-check" --> 02((02))
    02 -- "en" --> 1((1))
    1 -- "~en" --> 02
    1 -- "en" --> 1
  
```



enSeen
in every
state except
reset